

AI TechStack 2026

Assignment: Linear Regression, Optimization, and Feature Engineering

September 9, 2026

Overview and Learning Objectives

This assignment solidifies your understanding of supervised learning foundations, multidimensional gradient optimization, and empirical modeling. You will begin by formulating and manually calculating optimization steps on paper, implement fully vectorized algorithms inside a guided notebook, benchmark your results against production tools, and finally engineer features to compete on a Kaggle leaderboard.

The assignment consists of three parts:

- **Part 1 (Theoretical Calculations):** Manual derivations and optimization steps on paper.
 - **Part 2 (Guided Implementation):** Completing vectorized operations in `Task_Part2.ipynb` and validation.
 - **Part 3 (Competitive Leaderboard):** End-to-end regression pipeline design and leaderboard submission on Kaggle.
-

Part 1: Theoretical Foundations (Handwritten)

Complete the following problems with full mathematical steps on paper. Submit clear, legible scans.

Problem 1: Geometry of Optimization and Learning Rates

Consider an abstract objective surface parametrized by weights $\vec{w} = [w_1, w_2]^T$:

$$J(w_1, w_2) = 3w_1^2 - 6w_1 + w_2^2 - 4w_2 + 8$$

- Derive the analytical gradient vector $\nabla J(\vec{w})$.
- Find the exact coordinates of the stationary optimal point $\vec{w}^* = [w_1^*, w_2^*]^T$.
- Starting from $\vec{w}^{(0)} = [0, 0]^T$, perform **one update step** of Gradient Descent using vector notation under two learning rates:
 - Candidate A: $\alpha = 0.05$
 - Candidate B: $\alpha = 0.20$
- Compute the Euclidean distance of both candidates to the optimal point $\|\vec{w}^{(1)} - \vec{w}^*\|_2$. Briefly analyze why Candidate B achieves superior convergence speed without oscillating.

Problem 2: Multi-Feature Gradient Descent Manual Trace

You are tasked with building a linear regression model to predict vehicle braking distance (y) using two input features: normalized speed (x_1) and road surface factor (x_2), given the calibration data below:

Sample (i)	Speed (x_1)	Surface (x_2)	Stopping Distance (y)
1	1.0	0.0	2.0
2	2.0	1.0	5.0
3	0.0	1.0	3.0

Configuration: Use the standard Mean Squared Error (MSE) objective. Initialize all weights and the bias offset to zero ($\vec{w} = \vec{0}, b = 0$), and set the learning rate to $\alpha = 0.1$.

- Write down the explicit mathematical equations for the linear model hypothesis, the MSE cost function, and the simultaneous gradient update rules for weights $\vec{w}$ and bias b in matrix/vector notation.
- Trace the execution on paper and compute the updated parameters after **Epoch 1** and **Epoch 2**. State all intermediate sample predictions and residual errors clearly (maintain 4 decimal places).
- Evaluate the numerical value of the cost function before training begins and immediately after Epoch 2 to demonstrate that the loss is decreasing.

Part 2: Guided Implementation (Task_Part2.ipynb)

In the provided semi-prepared Jupyter notebook (Task_Part2.ipynb), complete the skeleton functions by filling in the blanks (---):

- Vectorized Computational Core:** Implement `compute_cost`, `compute_gradient`, and `gradient_descent` strictly using NumPy matrix primitives (`np.dot`, `@`). Explicit Python loops over training instances are strictly prohibited.
- Analytical Parity Check:** Pass the calibration dataset from Problem 2 into your implementation. Your printed outputs for Epoch 1 and Epoch 2 must match your handwritten results exactly.
- Feature Scaling & Non-Linear Fits:** Transform raw inputs into a degree-3 polynomial space, implement robust Z-score feature normalization, optimize parameters via your scratch engine, and plot both the loss convergence profile and the fitted curve.
- Scikit-Learn Alignment & Verification:** Replicate the training process using `LinearRegression` and `SGDRegressor`. Quantitatively verify that your scratch RMSE matches the library baseline within an absolute tolerance of 0.05.

Part 3: Competitive Leaderboard Challenge

In this section, you will tackle a real-world tabular dataset to predict used vehicle selling prices (`price`). No templates are provided; you must build your pipeline from scratch and compete on Kaggle.

Competition Access: Join the competition using the following official invitation link:

<https://www.kaggle.com/t/8dfb6f157f1c4803aededf8c13d93bb2>

Strict Model Constraints

To emphasize fundamental data processing and feature engineering, this competition operates under a **pure linear modeling restriction**:

- **Permitted Models: Only Linear Regression** is allowed (Standard Ordinary Least Squares, or Linear Regression regularized via L_1/L_2 penalty formulations).
- **Strictly Prohibited:** Tree-based algorithms (Decision Trees, Random Forests, XGBoost, LightGBM, CatBoost), Ensembles, and Neural Networks are **strictly forbidden**. Top leaderboard submissions will be audited, and any non-linear model will be disqualified.

Core Areas for Optimization

Your competitive performance depends entirely on data preparation and feature extraction:

- **Text Signal Extraction:** Parsing unstructured technical strings (`engine`, `transmission`) into continuous numerical signals (e.g., horsepower, engine displacement, transmission gear counts).
- **Feature Engineering:** Creating domain-specific indicators such as vehicle age, annual mileage wear rates, and title/accident status indicators.
- **Categorical Encoding:** Encoding high-cardinality metadata (`brand`, `model`, `fuel_type`) cleanly while avoiding severe dimensionality explosion or multicollinearity.
- **Regularization & Scaling:** Applying feature scaling (Z-score standardization) and tuning the regularization penalty parameter (λ) to control model variance and prevent overfitting.

Evaluation Metric & Baseline

Submissions are evaluated on the hidden test set using the R^2 **Score (Coefficient of Determination)**:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

We have placed an official baseline model on the leaderboard achieving an R^2 score of ~ 0.45676 . Your goal is to beat this benchmark.

Submission Instructions

All deliverables must be pushed directly to your designated assignment repository on **GitHub** under the following structured file layout:

- **Part1.Theoretical.Solutions.pdf:** A clear scan of your handwritten mathematical derivations and step-by-step traces for Problems 1 and 2.
- **Part2.Implementation.ipynb:** Your completed and executed notebook for Part 2 with all code blocks filled, cell outputs preserved, and plots visible.
- **Part3.Kaggle.Pipeline.ipynb:** Your complete, reproducible Kaggle pipeline notebook containing data cleaning, feature extraction, linear model training, and submission generation.